



1 Introducción

Muchas de las actividades que realizamos actualmente implican el acceso a recursos en sitios remotos. Consultar una página web, enviar un correo o participar de una sesión de juego en línea son actividades que requieren conectar al menos dos equipos, separados por metros o por miles de kilómetros. Para facilitar dicha conectividad, los computadores, celulares, consolas y otros terminales se agrupan en *redes*, que deben ser alcanzables entre sí.

En esta práctica los estudiantes se familiarizarán con el uso de la programación orientada a objetos (POO) aplicada a problemas de redes de computadores. Se abordará el modelamiento e implementación de enrutadores y redes, utilizando los contenedores de la STL (**vector**, **list**, entre otros) para representar estructuras de datos dinámicas. El enfoque principal consistirá en simular una red de enrutadores. Los **enrutadores** son equipos intermedios que se encargan de proveer conectividad entre las diferentes redes de Internet. Un enrutador puede conocer una o varias redes y compartir esta información con otros enrutadores para construir bases de datos consistentes y, así, llevar la información en forma de paquetes desde un origen hasta un destino. En esas bases de datos se guarda la información para encontrar el camino más corto (usualmente conocido como “el mejor camino”) a todas las redes de una topología, en la cual cada dispositivo debe ser capaz de construir y actualizar su tabla de enrutamiento en función de los cambios en la topología o del costo de los enlaces.

La actividad combina conceptos de programación avanzada con fundamentos de telecomunicaciones, proporcionando una visión práctica de cómo la informática y las redes se integran para respaldar la comunicación global.

2 Objetivos

- Adquirir destreza en el modelado e implementación de objetos.
- Adquirir destreza en el uso de los contenedores de la STL.
- Emplear POO para resolver problemas asociados a las redes de computadores.

3 Ejemplos

El profesor dará varios ejemplos de uso de contenedores, tales como **vector**, **list** y **map**, implementados en la STL.

4 Contexto

Consideremos una red de 4 enrutadores (A, B, C, D) conectados entre sí mediante enlaces con costos, como se muestra en la Figura 1. Los costos pueden representar la distancia, el dinero que cueste usarlos o una función de la velocidad del enlace, o incluso, una combinación de varios factores. Un camino es un conjunto de uno o más enlaces que conectan dos nodos. Por ejemplo, en la Figura 1, entre los nodos A y C hay varios caminos, $A \xrightarrow{10} C$, $A \xrightarrow{5} D \xrightarrow{2} C$ y $A \xrightarrow{4} B \xrightarrow{3} C$. El costo asociado a dichos caminos es

la suma de los costos de cada enlace; para los caminos mencionados anteriormente, los costos serían 10, 7 y 7, respectivamente. Cada enrutador debe calcular el camino más corto hacia los demás enrutadores de la topología.

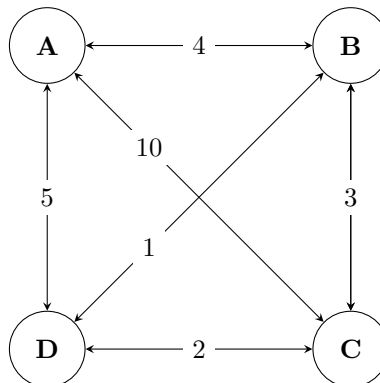


Figura 1: Red de enrutadores con sus costos de enlace.

Una vez que un enrutador se enciende, debe comenzar a construir una tabla de costos para cada destino. En un principio agrega los destinos directamente conectados (con los que tiene enlace directo) y asigna el costo de dicho enlace, por ejemplo para el caso del enrutador A:

	A	B	C	D
A	0	4	10	5

Para el enrutador B:

	A	B	C	D
B	4	0	3	1

Dichas tablas no representan los caminos más cortos para esa topología. Por ejemplo, el enrutador A tiene una ruta directa a C con costo 10, pero las rutas a través de D o de B tienen un costo de 7, lo que las convierte en una mejor opción. Dicho esto, después de intercambiar información con los demás enrutadores, se ejecuta un algoritmo que identifica y consigna los valores de las rutas más cortas. La tabla con los menores costos a cada destino en A quedaría:

	A	B	C	D
A	0	4	7	5

Es deseable (algunos algoritmos lo permiten) que cada enrutador guarde los caminos completos que trazan la ruta que lleva a cada destino, por ejemplo en A:

- Para llegar a B toma el camino $A \rightarrow B$
- Para llegar a C, toma el camino $A \rightarrow B \rightarrow C$
- Para llegar a D, toma el camino $A \rightarrow D$

Cada enrutador realiza el procedimiento y la tabla de costos para cada destino, y la tabla conjunta quedaría:

	A	B	C	D
A	0	4	7	5
B	4	0	3	1
C	7	3	0	2
D	5	1	2	0

Adicionalmente, cada enrutador puede guardar el camino completo para cada destino; así, cuando va a enviar información a un destino, sabe a través de qué nodo vecino debe enviar primero un paquete.

5 Ejercicio

En esta práctica se utilizará un modelo centralizado para simplificar el problema. En redes reales, cada enrutador ejecuta su propio algoritmo y construye su tabla de enrutamiento de forma distribuida.

Con base en el conocimiento adquirido en las sesiones teóricas y en la información de esta guía, y utilizando los conceptos de POO, escriba un programa que permita simular una red de enrutadores. El programa desarrollado debe incluir lo siguiente:

1. **Modelamiento de un enrutador:** cada enrutador debe representar únicamente la información local de la red. En particular, debe:
 - i) Almacenar su identificador (nombre).
 - ii) Almacenar sus vecinos directamente conectados y el costo de cada enlace en un contenedor adecuado de la STL.
 - iii) Almacenar su tabla de costos a otros destinos, la cual se actualizará con la información proporcionada por la red.
2. **Modelamiento de la red:** implemente una clase `Red` que represente la topología completa. Esta clase debe:
 - i) Almacenar el conjunto de enrutadores.
 - ii) Gestionar las conexiones entre enrutadores.
 - iii) Calcular los caminos entre pares de enrutadores.
 - iv) Actualizar las tablas de costos de cada enrutador cuando la topología cambie.
3. El programa debe permitir la agregación o eliminación de enrutadores y enlaces en tiempo de ejecución. Una vez realizado un cambio en la topología, la red debe recalcular los caminos y actualizar la información asociada a cada enrutador.
4. Configurar una red a partir de un archivo que describa la topología (enrutadores y conexiones). El estudiante determinará el formato a utilizar.
5. Dados los enrutadores (origen y destino), el programa debe:
 - i) Encontrar un camino entre ellos.
 - ii) Representar dicho camino utilizando un contenedor de la STL (por ejemplo, `list` o `vector`).
 - iii) Calcular e imprimir el costo asociado a ese camino.
6. Implementar las clases auxiliares necesarias para completar el programa. Se debe incluir el diagrama de clases, indicando:
 - i) Relaciones entre clases.
 - ii) Atributos y métodos principales.
7. **Uso de contenedores STL:** el estudiante debe seleccionar y justificar el uso de contenedores como `map`, `vector` o `list` en las diferentes partes del sistema (enrutadores, red, caminos, etc.).
8. **Robustez y validación:** El programa debe gestionar estados de error, tales como la inexistencia de caminos entre nodos (grafos no conexos), intentos de conexión a enrutadores inexistentes y manejo de archivos de topología mal formados.